

C++ COMPETITIVE PROGRAMMING + DATA STRUCTURES + ALGORITHMS MASTER TEXTBOOK

PART XVIII

Advanced Data Structures

Fenwick Tree, Segment Tree with lazy propagation, Sparse Table, and Binary Lifting / LCA — answering range and ancestor queries in $O(\log n)$ instead of $O(n)$

PART XVIII

Advanced Data Structures

One theme, four structures: precompute a tree-shaped summary so a range or ancestor question never needs a linear scan

Part XVII organized itself around never re-scanning text. This Part has the same spirit, applied to arrays and trees: a plain array answers a range-sum query in $O(n)$ and a range-update in $O(n)$. Every structure here trades a bit of preprocessing for $O(\log n)$ — or, for static data, $O(1)$ — per query. The right choice depends on exactly one question, asked repeatedly through this Part: does the data change after preprocessing, and do you need point updates or range updates?

Contents of Part XVIII

- 1. Fenwick Tree (Binary Indexed Tree)
- 2. Segment Tree — Build, Query, Update
- 3. Lazy Propagation
- 4. Sparse Table and Static RMQ
- 5. Binary Lifting — Fast Ancestor Queries
- 6. LCA via Binary Lifting
- Part XVIII Revision, Cheat Sheet, and Interview Questions

Chapter 1 — Fenwick Tree (Binary Indexed Tree)

1.1 The problem it solves

Operation	Plain array	Fenwick Tree
Point update (add x to index i)	$O(1)$	$O(\log n)$
Prefix sum query (sum of index 0..i)	$O(n)$	$O(\log n)$
Range sum query (sum of index l..r)	$O(n)$	$O(\log n)$, via two prefix queries

A prefix-sum array alone answers queries in $O(1)$ but takes $O(n)$ to update, since every later prefix shifts. A Fenwick Tree balances both at $O(\log n)$ each — the standard choice whenever updates and prefix queries are interleaved.

1.2 The core idea: each index owns a range, sized by its lowest set bit

Index i in the Fenwick array stores the sum of a range of the original array, ending at i , whose length is exactly $i \& (-i)$ — the value of i 's lowest set bit (Part XVI's bit trick). This single fact drives both operations.

i (1-indexed)	binary	$i \& (-i)$	range this index covers
1	0001	1	covers just index 1
2	0010	2	covers indices 1..2
3	0011	1	covers just index 3
4	0100	4	covers indices 1..4
6	0110	2	covers indices 5..6
8	1000	8	covers indices 1..8

1.3 Update and query, in code

```
vector<long long> bit;    // 1-indexed, size n+1
int N;

void update(int i, long long delta) {    // point update: add
    delta at i
    for (; i <= N; i += i & (-i))
        bit[i] += delta;
}

long long prefixSum(int i) {              // sum of [1..i]
    long long sum = 0;
    for (; i > 0; i -= i & (-i))
        sum += bit[i];
    return sum;
}

long long rangeSum(int l, int r) {        // sum of [l..r],
    l-indexed
    return prefixSum(r) - prefixSum(l - 1);
}
```

1.4 Why the loops move the way they do

Operation	Direction	Step	Why
update(i, delta)	i increases	i += i & (-i)	moves to the next index whose range INCLUDES i
prefixSum(i)	i decreases	i -= i & (-i)	jumps back by exactly the range the current index covers

1.5 Dry run: array [3, 2, -1, 6, 5, 4, -3, 3] (1-indexed)

Building via 8 individual update() calls, then querying prefixSum(5):

Call	i sequence visited	Effect
update(1, 3)	1 → 2 → 4 → 8	bit[1], bit[2], bit[4], bit[8] each += 3
update(5, 5)	5 → 6 → 8	bit[5], bit[6], bit[8] each += 5

Call	i sequence visited	Effect
prefixSum(5)	5 → 4 → 0 (stop)	sum = bit[5] + bit[4]

 0-indexed data still needs a 1-indexed Fenwick array

The i & $(-i)$ trick relies on index 0 being meaningless (0 & (-0) loops forever). The standard convention: store data 1-indexed inside the Fenwick array, shifting by +1 when converting from a 0-indexed input array.

Practice — Chapter 1

Problem 1.1 — Count inversions using a Fenwick Tree [Level 3]

Count the number of pairs (i, j) with $i < j$ and $a[i] > a[j]$.

Hint 1: Compress values to ranks $1..n$. Process the array right to left, querying `prefixSum(rank-1)` for how many SMALLER elements have been seen so far, then `update(rank, 1)`.

Key observation: A Fenwick Tree used as a frequency array turns 'how many smaller elements appeared already' into an $O(\log n)$ prefix-sum query instead of an $O(n)$ scan — this is the single most common Fenwick Tree application after plain range sums.

Complexity: $O(n \log n)$, versus $O(n^2)$ brute force.

Problem 1.2 — Range update, point query [Level 3]

Support adding a value to an entire range $[l, r]$, and querying a single index's current value, both in $O(\log n)$.

Hint 1: Use a Fenwick Tree over the DIFFERENCE array (Part I hints at this idea): `update(l, +delta)` and `update(r+1, -delta)`; a point query is just `prefixSum(i)` on the difference-array Fenwick Tree.

Key observation: This flips the Fenwick Tree's usual roles — the difference-array trick from prefix sums (Part III) combines with the Fenwick Tree to swap which operation (update vs. query) is the 'range' one.

Complexity: $O(\log n)$ per range update and per point query.

Chapter 2 — Segment Tree: Build, Query, Update

2.1 Why a Fenwick Tree isn't always enough

Need	Fenwick Tree	Segment Tree
Range sum, point update	Yes — simplest choice	Yes, but more code
Range minimum/maximum query	Awkward (no clean inverse operation)	Yes, naturally
Range update + range query together	Possible with tricks	Yes, naturally (with lazy propagation, Ch. 3)
Arbitrary associative combine function	No	Yes — the general-purpose choice

✓ Rule of thumb

If it's plain range-sum with point updates, reach for a Fenwick Tree — it's shorter and faster in practice. For range minimum/maximum, range updates, or anything beyond addition, reach for a segment tree.

2.2 The idea: a binary tree over array ranges

Each node of a segment tree represents a contiguous range of the array. The root covers the whole array; each node's two children split its range in half. A leaf covers a single element. Stored in an array (like a heap, Part XII), node 1 is the root, and node i 's children are $2i$ and $2i+1$.

```
vector<long long> seg; // size 4*n is a safe upper bound
int N;

void build(vector<int>& a, int node, int start, int end) {
    if (start == end) { seg[node] = a[start]; return; }
    int mid = (start + end) / 2;
    build(a, 2*node, start, mid);
    build(a, 2*node+1, mid+1, end);
    seg[node] = seg[2*node] + seg[2*node+1]; // combine step
                                           (sum here)
}
```

2.3 Range query

```
long long query(int node, int start, int end, int l, int r) {
    if (r < start || end < l) return 0; // no
    overlap
    if (l <= start && end <= r) return seg[node]; // total
    overlap
    int mid = (start + end) / 2;
    return query(2*node, start, mid, l, r) + // partial
    overlap:
        query(2*node+1, mid+1, end, l, r); // recurse
        both sides
}
// call as: query(1, 0, n-1, l, r)
```

Case	Condition	Action
No overlap	the node's range is entirely outside [l, r]	contribute nothing (return the identity, 0 for sum)
Total overlap	the node's range is entirely inside [l, r]	return the node's precomputed value directly — no further recursion
Partial overlap	the node's range straddles the boundary	recurse into both children and combine

2.4 Point update

```
void update(int node, int start, int end, int idx, int val) {
    if (start == end) { seg[node] = val; return; }    // reached
    the leaf
    int mid = (start + end) / 2;
    if (idx <= mid) update(2*node, start, mid, idx, val);
    else update(2*node+1, mid+1, end, idx, val);
    seg[node] = seg[2*node] + seg[2*node+1];        //
    recombine on the way up
}
```

2.5 Why every operation is $O(\log n)$

Build visits every node once — $O(n)$ total (a segment tree has roughly $2n-1$ nodes). Query and update each follow a path down the tree; a query additionally branches into at most two paths per level at the boundaries, but never more — bounded by $O(\log n)$ total work, the same halving argument from Part X and Part VI.

⚠ The $4n$ array-size convention

Sizing the underlying array as $4*n$ (not $2n$ or a tightly computed bound) is a defensive, widely used habit — it comfortably covers the worst-case node count for any n without needing to prove a tighter bound under contest time pressure.

Practice — Chapter 2

Problem 2.1 — Range minimum query (RMQ) with point updates [Level 2]

Adapt the segment tree above to answer range-minimum queries instead of range-sum.

Hint 1: Change the combine step from $+$ to $\min$, and the 'no overlap' identity from 0 to $+\infty$ (or `INT_MAX`).

Approach: Every function stays structurally identical — only the combine operation and identity element change.

Complexity: $O(\log n)$ build/query/update, $O(n)$ overall build.

Problem 2.2 — Count of a specific value in a range [Level 3]

Given an array, answer queries: how many times does value v appear in range $[l, r]$? (Point updates allowed.)

Hint 1: Build a separate segment tree per distinct value is too much memory for large alphabets — instead consider a merge-sort-tree style structure, or, if values are small, one Fenwick Tree per value.

Key observation: This is a preview of 'segment tree of something more complex than a number' — nodes can store sorted lists, frequency maps, or other structures, not just single numbers.

Complexity: Varies by approach; $O(\log^2 n)$ per query with a merge-sort tree.

Chapter 3 — Lazy Propagation

3.1 The problem: range updates are slow without it

Chapter 2's segment tree updates a single index in $O(\log n)$. Updating an entire range $[l, r]$ naively means calling that update $r - l + 1$ times — $O((r-l+1) \log n)$, which degrades to $O(n \log n)$ for a full-array update. Lazy propagation makes range updates $O(\log n)$ too, by deferring work.

	Point update	Range update	Range query
Plain segment tree (Ch. 2)	$O(\log n)$	$O((r-l+1) \times \log n)$	$O(\log n)$
Segment tree + lazy propagation	$O(\log n)$	$O(\log n)$	$O(\log n)$

3.2 The idea: mark a subtree "pending" instead of updating it immediately

When a range update fully covers a node's range, that node's own value is updated immediately — but its children are NOT recursed into right away. Instead, a `lazy[node]` marker records “this subtree owes an update,” and it's only pushed down to the children the next time something actually needs to visit them.

```

vector<long long> seg, lazy;

void pushDown(int node, int start, int end) {
    if (lazy[node] == 0) return;           // nothing
    pending
    int mid = (start + end) / 2;
    // apply the pending update to both children, and queue it
    further down
    seg[2*node] += lazy[node] * (mid - start + 1);
    seg[2*node+1] += lazy[node] * (end - mid);
    lazy[2*node] += lazy[node];
    lazy[2*node+1] += lazy[node];
    lazy[node] = 0;                       // cleared:
    now applied to children
}

void updateRange(int node, int start, int end, int l, int r,
long long val) {
    if (r < start || end < l) return;       // no
    overlap
    if (l <= start && end <= r) {           //
    total overlap
        seg[node] += val * (end - start + 1);
        lazy[node] += val;
        return;                           //
        STOP -- don't recurse further
    }
    pushDown(node, start, end);             //
    partial overlap: must descend
    int mid = (start + end) / 2;
    updateRange(2*node, start, mid, l, r, val);
    updateRange(2*node+1, mid+1, end, l, r, val);
    seg[node] = seg[2*node] + seg[2*node+1];
}

```

3.3 Query must also push down

```

long long queryRange(int node, int start, int end, int l, int r)
{
    if (r < start || end < l) return 0;
    if (l <= start && end <= r) return seg[node];
    pushDown(node, start, end);           // flush any pending
    update first
    int mid = (start + end) / 2;
    return queryRange(2*node, start, mid, l, r) +
           queryRange(2*node+1, mid+1, end, l, r);
}

```

⚠ The rule that makes lazy propagation correct

Any time code is about to READ or MODIFY a node's children, it must call `pushDown` first. Skipping this on any code path (a common bug when adding new query types later) means children can silently hold stale values.

3.4 Dry run: range-add then range-query

Array `[1,1,1,1]`, `updateRange` to add 5 to indices `[1,2]` (0-indexed), then `queryRange(0,3)`:

Step	What happens
<code>updateRange([1,2], +5)</code> reaches the node covering <code>[1,2]</code> exactly	<code>seg[node] += 5×2=10</code> immediately; <code>lazy[node] = 5</code> ; recursion STOPS here — children of this node are never visited
<code>queryRange(0,3)</code> later needs to look inside <code>[1,2]</code> 's parent range	<code>pushDown</code> fires: children's <code>seg[]</code> values are corrected using <code>lazy=5</code> , then <code>lazy</code> is cleared on this node
Final query result	$1 + (1+5) + (1+5) + 1 = 14$ — correct, and the expensive per-element update never happened

 Quick Revision

- Lazy propagation defers applying an update to a node's children until they're actually needed.
- A total-overlap range update stops immediately after marking `lazy[node]` — it never recurses into children that don't need to be touched yet.
- Both `updateRange` and `queryRange` must call `pushDown` before descending into a node's children on a partial-overlap path.
- This turns $O((r-l+1) \log n)$ naive range updates into $O(\log n)$, matching point-update speed.

Practice — Chapter 3

Problem 3.1 — Range add, range sum [Level 4]

Support adding a value to every element in $[l, r]$, and querying the sum of $[l, r]$, both in $O(\log n)$.

Hint 1: This is exactly the code shown in this chapter — implement it directly and dry-run it on a small array by hand.

Complexity: $O(\log n)$ per update and query.

Problem 3.2 (harder) — Range assign (set), range max [Level 5]

Support setting every element in $[l, r]$ to a fixed value v , and querying the maximum of $[l, r]$.

Hint 1: The lazy marker now needs to distinguish 'no pending update' from 'pending assignment to 0' — a boolean `hasLazy` flag alongside the lazy value is the standard fix.

Key observation: Range-assign lazy propagation differs from range-add: `pushDown` must **OVERWRITE** the children's values rather than add to them, and a pending assignment at a node makes any older pending lazy value at that node irrelevant (the newer assignment simply replaces it).

Complexity: $O(\log n)$ per update and query.

Chapter 4 — Sparse Table and Static RMQ

4.1 When updates are off the table entirely

If the array never changes after preprocessing, a segment tree's $O(\log n)$ per query can be beaten: a sparse table answers a range minimum (or maximum, GCD, AND, OR — any idempotent operation) query in $O(1)$, after $O(n \log n)$ preprocessing.

	Preprocessing	Query	Supports updates?
Segment tree	$O(n)$	$O(\log n)$	Yes
Sparse table	$O(n \log n)$	$O(1)$	No — static only

4.2 Idempotent: why this only works for certain operations

An operation is idempotent if combining overlapping ranges doesn't corrupt the answer — $\min(\min(a, b), b)$ still equals $\min(a, b)$, so counting an element twice does no harm. Sum is NOT idempotent — counting a range twice inflates the total — which is exactly why sparse tables handle min/max/gcd/and/or well but not sum.

Operation	Idempotent?	Sparse table works?
min / max	Yes	Yes
gcd	Yes	Yes
bitwise AND / OR	Yes	Yes
sum	No	No — use Fenwick/segment tree instead
xor	No	No

4.3 The idea: precompute every power-of-two-length range

`sparse[k][i]` stores the minimum of the range starting at `i` with length 2^k . Any range `[l, r]` can be covered by exactly two (possibly overlapping) power-of-two ranges — one starting at `l`, one ending at `r` — computed once in $O(n \log n)$ and reused for $O(1)$ queries.

```
int LOG[MAXN];
int sparse[LOG_MAX][MAXN];

void build(vector<int>& a) {
    int n = a.size();
    for (int i = 2; i <= n; i++) LOG[i] = LOG[i/2] + 1;
    for (int i = 0; i < n; i++) sparse[0][i] = a[i];
    for (int k = 1; (1 << k) <= n; k++)
        for (int i = 0; i + (1 << k) <= n; i++)
            sparse[k][i] = min(sparse[k-1][i], sparse[k-1][i +
                               (1 << (k-1))]);
}

int query(int l, int r) { // inclusive, 0-indexed
    int k = LOG[r - l + 1];
    return min(sparse[k][l], sparse[k][r - (1 << k) + 1]);
}
```

4.4 Dry run: why overlapping ranges are safe here

Array indices `[1..10]`, query range `[2, 9]` (length 8). $k = \text{LOG}[8] = 3$, so $2^3 = 8$ -length ranges are used:

Range used	Covers indices	Overlap with the other range
<code>sparse[3][2]</code>	2..9 (length 8)	—
<code>sparse[3][9-8+1]=sparse[3][2]</code>	2..9 (length 8)	identical range in this exact case (8 divides evenly)

For a range whose length isn't a clean power of two (say length 6, $k=2$, so two length-4 ranges are used), the two chosen ranges genuinely overlap in the middle — and because `min` is idempotent, that overlap changes nothing about the correctness of the answer.

✓ Recognizing when a sparse table is the right call

Clue words: “no updates,” “static array,” “many RMQ queries.” If the problem ever mentions updates interleaved with queries, a sparse table is the wrong tool — fall back to a segment tree.

Practice — Chapter 4

Problem 4.1 — Range GCD queries [Level 2]

Given a static array, answer many range-GCD queries in $O(1)$ each after preprocessing.

Hint 1: gcd is idempotent ($\text{gcd}(\text{gcd}(a,b),b) = \text{gcd}(a,b)$) — identical structure to the RMQ sparse table, just swap min for gcd.

Complexity: $O(n \log n)$ preprocessing, $O(1)$ per query.

Chapter 5 — Binary Lifting: Fast Ancestor Queries

5.1 The problem

Given a rooted tree, answer “what is the k -th ancestor of node v ?” repeatedly, for many different v and k . Walking up one parent at a time costs $O(k)$ per query — too slow when k and the number of queries are both large. Binary lifting answers each query in $O(\log k)$ after $O(n \log n)$ preprocessing.

Approach	Preprocessing	Per query
Walk up one parent at a time	$O(n)$ (store direct parents)	$O(k)$
Binary lifting	$O(n \log n)$	$O(\log k)$

5.2 The idea: precompute 2^0 -th, 2^1 -th, 2^2 -th... ancestors

$up[k][v]$ stores the 2^k -th ancestor of node v . Any k can be written as a sum of powers of two (its binary representation) — Part XVI's binary exponentiation idea, applied to tree ancestors instead of numeric powers.

```

int up[LOG][MAXN];          // up[k][v] = 2^k-th ancestor of v
int depth[MAXN];

void buildBinaryLifting(int root, vector<vector<int>>& adj, int n) {
    // first: BFS/DFS to fill up[0][v] = direct parent, and
    // depth[v]
    // ... (standard tree traversal, omitted for brevity)

    for (int k = 1; k < LOG; k++)
        for (int v = 0; v < n; v++)
            up[k][v] = (up[k-1][v] == -1) ? -1 :
                up[k-1][up[k-1][v]];
}

int kthAncestor(int v, int k) {
    for (int i = 0; i < LOG && v != -1; i++)
        if (k & (1 << i)) v = up[i][v];
    return v;
}

```

5.3 Why $up[k][v] = up[k-1][up[k-1][v]]$

The 2^k -th ancestor is the 2^{k-1} -th ancestor of the 2^{k-1} -th ancestor — jumping halfway, twice. This is exactly the doubling idea from Part X's $O(\log n)$ power function, applied to “jump up the tree” instead of “multiply.”

5.4 Dry run: finding the 5th ancestor

5 in binary is 101 — bits set at positions 0 and 2. `kthAncestor(v, 5)` therefore jumps by $2^0=1$, then by $2^2=4$:

i	bit i of 5 (101)	Action
0	1	$v = up[0][v]$ — jump up 1
1	0	skip — no jump
2	1	$v = up[2][v]$ — jump up 4 (total so far: $1+4=5$)

Total distance climbed: $1 + 4 = 5$, matching k exactly — every value of k decomposes uniquely into powers of two this way, which is why the loop always lands exactly on the right ancestor.

Quick Revision

- Binary lifting precomputes 2^k -th ancestors for every node and every power of two up to $\log(n)$.
- $up[k][v] = up[k-1][up[k-1][v]]$ — doubling, the same idea as fast exponentiation (Part XVI) and merge sort's halving (Part X).
- Any k decomposes into a sum of powers of two (its binary representation) — walk that decomposition bit by bit to reach the exact k -th ancestor in $O(\log k)$.

Practice — Chapter 5

Problem 5.1 — Kth ancestor queries [Level 3]

Given a rooted tree and q queries of the form (v, k) , answer the k -th ancestor of v for each.

Hint 1: Build $up[][]$ once in $O(n \log n)$, then answer each query in $O(\log k)$ using the code in this chapter.

Complexity: $O(n \log n)$ preprocessing, $O(\log k)$ per query.

Chapter 6 — LCA via Binary Lifting

6.1 The problem

Given a rooted tree and two nodes u, v , find their Lowest Common Ancestor — the deepest node that is an ancestor of both. Part XI solved this for a general tree in $O(h)$ per query using recursive search from the root. Binary lifting answers it in $O(\log n)$ per query, which matters when there are many queries on a large, possibly unbalanced tree.

Approach	Preprocessing	Per query
Recursive search from root (Part XI)	$O(n)$	$O(h)$ — $O(n)$ worst case on a skewed tree
Binary lifting LCA	$O(n \log n)$	$O(\log n)$, always

6.2 The two-phase idea

Phase	Goal	How
1. Equalize depth	Bring the deeper of u, v up to the same depth as the other	Binary lifting jump by $(\text{depth}[u] - \text{depth}[v])$, using the bit-decomposition trick from Chapter 5
2. Climb together	Find the LAST point where u and v are still different, then step up once more	Try jumping both u and v by decreasing powers of two SIMULTANEOUSLY , only committing a jump if it keeps them distinct

```

int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int diff = depth[u] - depth[v];
    for (int i = 0; i < LOG; i++)                // Phase 1:
        equalize depth
        if (diff & (1 << i)) u = up[i][u];

    if (u == v) return u;                        // v was
    already an ancestor of u

    for (int i = LOG - 1; i >= 0; i--) {          // Phase 2:
        climb together
        if (up[i][u] != up[i][v]) {              // still
            different: safe to jump
            u = up[i][u];
            v = up[i][v];
        }
    }
    return up[0][u];                            // one more
    step reaches the LCA
}

```

Why Phase 2 goes from LARGE powers down to small

Jumping by the biggest safe power of two first, then progressively smaller ones, is the same greedy-from-the-top-bit idea used in binary representations generally: it guarantees u and v end up exactly one step below their LCA, using the fewest possible jumps, without ever overshooting past the LCA into unrelated territory.

6.3 Dry run

Tree: root 1, depth 0. Chain $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ (each depth+1), and separately $1 \rightarrow 6 \rightarrow 7$. Find LCA(5, 7): $\text{depth}[5]=4$, $\text{depth}[7]=2$.

Step	State
Phase 1: $\text{diff} = 4 - 2 = 2$	$u=5$ jumps up by 2 (bit 1 of "10") $\rightarrow u=3$, now $\text{depth}[u]=\text{depth}[v]=2$
Check $u == v$?	$u=3, v=7$ — not equal, continue to Phase 2

Step	State
Phase 2, large power first	$up[?][3]$ and $up[?][7]$ are checked; both eventually lead to node 1, so no jump is taken while they'd still match at node 1
Phase 2, smallest power (2^0)	$up[0][3]=1$, $up[0][7]=1$ — same, so still no jump (jumping would overshoot)
Final step	return $up[0][u] = up[0][3] = 1$

$LCA(5, 7) = 1$, correctly identified as the deepest common ancestor (the root, since the two chains only share the root).

Quick Revision

- Binary lifting LCA: equalize depth first (Phase 1), then climb both nodes together using decreasing powers of two, jumping only while it keeps them distinct (Phase 2).
- $O(n \log n)$ preprocessing enables $O(\log n)$ per query, beating recursive search's $O(h)$ worst case on unbalanced trees.
- The same $up[][]$ table built for k-th ancestor queries (Chapter 5) powers LCA directly — one preprocessing pass, two different query types.

Practice — Chapter 6

Problem 6.1 — Distance between two nodes in a tree [Level 3]

Given a rooted tree, answer queries for the number of edges on the path between u and v .

Hint 1: $distance(u, v) = depth[u] + depth[v] - 2 * depth[lca(u, v)]$.

Key observation: This formula works because the path from u to v goes up to the LCA and back down — the LCA's depth is subtracted twice since both partial paths count it once.

Complexity: $O(\log n)$ per query, given precomputed $depth[]$ and $up[][]$.

Part XVIII — Master Revision

Which Structure Should I Use?

Situation	Structure
Range sum + point updates	Fenwick Tree (simplest, fastest)
Range min/max, or range updates too, or a custom combine function	Segment Tree
Range updates AND range queries, both needed	Segment Tree + lazy propagation
Static array (no updates), many range min/max/gcd queries	Sparse Table — $O(1)$ per query
Many k-th ancestor queries on a tree	Binary Lifting
Many LCA queries on a large or unbalanced tree	Binary Lifting LCA

Must Remember

Quick Revision

- Fenwick Tree: $i \& (-i)$ isolates the lowest set bit, defining exactly which range each index owns; update moves UP by adding it, query moves DOWN by subtracting it.
- Segment Tree: no-overlap / total-overlap / partial-overlap are the three cases every query and update must handle; total overlap is what makes it $O(\log n)$ instead of $O(n)$.
- Lazy propagation defers updates to children until pushDown is called — both query and update must pushDown before descending on any partial-overlap path.
- Sparse Table only works for idempotent operations (min, max, gcd, AND, OR) — never for sum or xor — and only on static (unchanging) data.
- Binary Lifting: $up[k][v] = up[k-1][up[k-1][v]]$, the same doubling idea as fast exponentiation; any k decomposes into powers of two to reach it in $O(\log k)$.
- Binary Lifting LCA: equalize depth, then climb together using decreasing powers of two, only jumping while the two nodes stay distinct.

Common Mistakes Recap

Mistake	Fix
Using a Fenwick Tree 0-indexed	$i \& (-i)$ breaks at $i=0$ — always store data 1-indexed internally
Sizing a segment tree array as $2n$ instead of $4n$	Use $4n$ as a safe, standard upper bound on node count
Forgetting pushDown before descending into children (lazy propagation)	Call pushDown at the START of any function that's about to recurse into children on a partial-overlap path
Using a sparse table for range SUM queries	Sum isn't idempotent — overlapping ranges double-count; use Fenwick/segment tree instead
Confusing depth (Part XI: distance from root) direction in binary lifting	$up[][]$ always moves TOWARD the root, i.e., depth decreases

Mistake	Fix
Jumping too far in LCA's Phase 2 and overshooting past the LCA	Only commit a jump when $up[i][u] \neq up[i][v]$ — that's what prevents overshooting

Complexity Table

Structure	Preprocessing	Query	Update
Fenwick Tree	$O(n \log n)$ (n updates)	$O(\log n)$	$O(\log n)$
Segment Tree	$O(n)$	$O(\log n)$	$O(\log n)$
Segment Tree + lazy propagation	$O(n)$	$O(\log n)$	$O(\log n)$ range update
Sparse Table	$O(n \log n)$	$O(1)$	not supported
Binary Lifting (k-th ancestor)	$O(n \log n)$	$O(\log k)$	—
Binary Lifting (LCA)	$O(n \log n)$	$O(\log n)$	—

Interview-Style Questions

- Why does $i \& (-i)$ correctly determine the range each Fenwick Tree index is responsible for?
- Explain the three overlap cases (no/total/partial) in a segment tree query, and why total overlap is what makes the algorithm $O(\log n)$.
- Why must both query and update in a lazily-propagated segment tree call `pushDown` before recursing into children?
- Why does a sparse table work for range minimum but not for range sum?
- Why does binary lifting's $up[k][v] = up[k-1][up[k-1][v]]$ recurrence achieve $O(\log k)$ ancestor queries instead of $O(k)$?

- In binary lifting LCA, why does Phase 2 iterate from the largest power of two down to the smallest, rather than the other way around?

Cheat Sheet — Part XVIII

```
// Fenwick Tree
void update(int i, long long d){ for(;i<=N;i+=i&(-i)) bit[i]+=d;
}
long long prefixSum(int i){ long long s=0; for(;i>0;i-=i&(-i))
s+=bit[i]; return s; }

// Segment Tree (sum) query cases
// no overlap: r<start||end<l -> return identity
// total overlap: l<=start&&end<=r -> return seg[node]
// partial: recurse both children, combine

// Lazy propagation: pushDown BEFORE descending into children
void pushDown(int node,int start,int end){
    if(!lazy[node]) return;
    int mid=(start+end)/2;
    seg[2*node]+=lazy[node]*(mid-start+1);
    seg[2*node+1]+=lazy[node]*(end-mid);
    lazy[2*node]+=lazy[node]; lazy[2*node+1]+=lazy[node];
    lazy[node]=0;
}
```

```

// Sparse Table (min)
for (int k=1; (1<<k)<=n; k++)
    for (int i=0; i+(1<<k)<=n; i++)
        sparse[k][i] = min(sparse[k-1][i],
                           sparse[k-1][i+(1<<(k-1))]);
int query(int l,int r){ int k=LOG[r-l+1]; return
min(sparse[k][l], sparse[k][r-(1<<k)+1]); }

// Binary Lifting
for (int k=1;k<LOG;k++) for (int v=0;v<n;v++)
    up[k][v] = up[k-1][v]==-1 ? -1 : up[k-1][up[k-1][v]];

int lca(int u,int v){
    if (depth[u]<depth[v]) swap(u,v);
    int diff=depth[u]-depth[v];
    for (int i=0;i<LOG;i++) if(diff&(1<<i)) u=up[i][u];
    if (u==v) return u;
    for (int i=LOG-1;i>=0;i--) if(up[i][u]!=up[i][v]){
        u=up[i][u]; v=up[i][v]; }
    return up[0][u];
}

```

Coming up in Part XIX

Part XIX — Advanced Algorithms covers divide and conquer, meet in the middle, coordinate compression, sweep line, and the small-to-large technique — algorithmic strategies that sit on top of the data structures from this Part rather than replacing them. Several of that Part's techniques (sweep line especially) reach directly for a Fenwick or segment tree as their core engine.